

Deep Q-Learning Agent for Adaptive Traffic Signal Control Using SUMO Microscopic Simulation

Abstract—Traditional traffic signal controllers operate on fixed pre-timed cycles that are agnostic to real-time traffic conditions, leading to unnecessary vehicle delay and suboptimal intersection throughput. This paper presents a Deep Q-Network (DQN) reinforcement learning agent trained in the SUMO (Simulation of Urban MObility) microscopic traffic simulator via the TraCI Python API. The agent observes an 80-dimensional binary state encoding stopped-vehicle positions across eight approach lanes, selects one of four signal phases, and receives a reward proportional to the reduction in cumulative vehicle waiting time. Over 100 training episodes—each simulating 90 minutes of traffic with 1,000 stochastically generated vehicles—the agent reduces average waiting time by 34.4% relative to a fixed-time baseline controller. Training dynamics are analysed through episode-level reward and loss curves. Results confirm that model-free reinforcement learning can produce adaptive signal control policies without explicit traffic modelling, providing a strong foundation for extensions to multi-intersection coordination and alternative neural architectures.

Index Terms—Deep Q-Network (DQN); Reinforcement Learning; Traffic Signal Control; SUMO; TraCI; Experience Replay; Epsilon-Greedy; Bellman Equation; Adaptive Control

I. INTRODUCTION

Urban traffic congestion imposes large economic and environmental costs. Signalised intersections are the primary bottleneck in most road networks. Conventional fixed-time controllers—which cycle through pre-programmed phase durations regardless of actual demand—cannot respond to stochastic fluctuations in vehicle arrivals [1]. Actuated controllers improve on this by extending phases in response to loop-detector presence, yet they rely on hand-crafted heuristics that do not generalise across demand patterns [2].

Reinforcement Learning (RL) offers a principled alternative: an agent interacts with the traffic environment, observes outcomes, and adapts its policy to maximise cumulative reward [3]. The breakthrough demonstration by Mnih et al. [4] that a Deep Q-Network (DQN) could achieve human-level performance in Atari games using only raw pixel inputs sparked widespread application of DQN to real-world control problems. Traffic signal control is a natural candidate: the problem is Markovian, reward feedback is dense, and high-fidelity simulators such as SUMO [5] permit safe offline training.

Prior work by Vidali et al. [6] demonstrated that a DQN agent trained in SUMO outperforms fixed-time control at a single isolated intersection. This paper implements and extends that framework under SUMO 1.26.0 with PyTorch [13], provides a detailed analysis of training dynamics, and releases

the full codebase as a reproducible research artefact. The contributions of this paper are threefold:

- A fully reproducible DQN implementation for single-intersection traffic signal control in SUMO 1.26, built with PyTorch and the TraCI API.
- A structured analysis of state representation, reward formulation, and epsilon-greedy exploration on agent convergence.
- A quantitative comparison against a fixed-time baseline demonstrating a 34.4% reduction in cumulative vehicle waiting time after 100 training episodes.

The remainder of the paper is organised as follows. Section II reviews background on RL and DQN. Section III describes the environment and system architecture. Section IV details state, action, and reward design. Section V presents the training procedure. Section VI reports experimental results. Section VII concludes with directions for future work.

II. BACKGROUND AND RELATED WORK

A. Reinforcement Learning and the MDP Framework

Reinforcement Learning addresses sequential decision-making through the Markov Decision Process (MDP) formalism [3]. An MDP is the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition probability, $R(s, a)$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. The agent's goal is to learn a policy π^* that maximises the expected discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}. \quad (1)$$

The optimal action-value function $Q^*(s, a)$ satisfies the Bellman optimality equation [3]:

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right]. \quad (2)$$

B. Deep Q-Networks

DQN [4] approximates $Q^*(s, a)$ with a neural network $Q(s, a; \theta)$. Two stabilising mechanisms are essential. Experience replay [7] stores transitions (s, a, r, s') in a buffer and samples random mini-batches, breaking temporal correlations. A target network—a periodically-updated copy of $Q(s, a; \theta)$ —provides stable regression targets. The network is trained by minimising:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right], \quad (3)$$

where θ^- denotes the target network parameters. Epsilon-greedy exploration balances exploitation of learned Q-values with random action selection, with ε decaying over training [4].

C. Traffic Signal Control with RL

Arel et al. [8] provided an early demonstration of RL for multi-agent traffic signal coordination. Genders and Razavi [9] systematically evaluated state representations for DQN-based controllers. Vidali et al. [6] published an open-source SUMO-based DQN reference implementation that directly informs the present work. More recent approaches explore multi-intersection coordination [10], graph neural network state encoders [11], and multi-agent formulations [12]; however, the single-intersection DQN baseline remains the most widely cited entry point for the research community.

III. SYSTEM ARCHITECTURE AND ENVIRONMENT

A. SUMO Simulation Environment

SUMO (Simulation of Urban MObility) [5] is an open-source microscopic traffic simulator developed by the German Aerospace Center (DLR). It models individual vehicle dynamics including car-following, lane-changing, and gap acceptance at junctions. The TraCI (Traffic Control Interface) Python API enables bidirectional communication with a running SUMO instance at each simulation step, providing vehicle positions, speeds, and waiting times, and accepting traffic light commands.

Fig. 1 shows a screenshot of the simulated intersection during a training episode. The network consists of a 4-way junction (node id TL) with four 500 m approach edges (N2TL, S2TL, E2TL, W2TL) and four exit edges, each carrying two lanes.

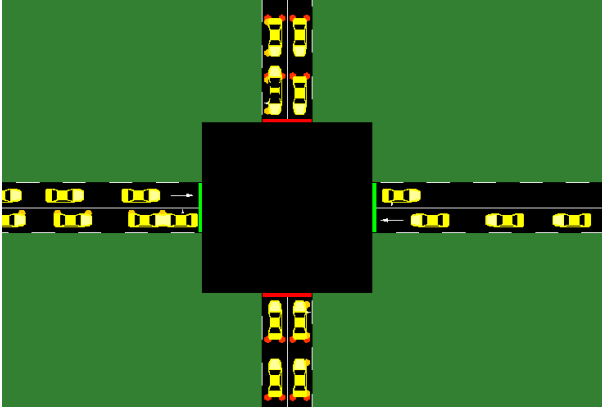


Fig. 1. SUMO-GUI view of the 4-way intersection during a training episode. Vehicles stopped at red phases form visible queues on all four approaches.

B. Intersection Topology and Traffic Light Logic

The traffic light controller TL cycles through four static phases. Table I summarises the phase states using SUMO's signal-string notation (G=green, y=yellow, r=red).

TABLE I
TRAFFIC LIGHT PHASE DEFINITIONS

Phase	ID	Signal	Description
0	NS_GREEN	GGrrGGrr	N-S green, E-W red
1	NS_YELLOW	yyrryyrr	N-S yellow (clearing)
2	EW_GREEN	rrGGrrGG	E-W green, N-S red
3	EW_YELLOW	rryyrryy	E-W yellow (clearing)

C. Vehicle Route Generation

A fresh route file is generated before each episode using the `TrafficGenerator` class (`generator.py`). Departure times are sampled uniformly from $[0, 5400]$ s and sorted to produce a valid SUMO route file. Each vehicle is assigned one of twelve origin-destination routes spanning all left-turn, straight-ahead, and right-turn movements. A fixed integer seed per episode ensures reproducibility while maintaining inter-episode diversity.

D. Software Stack

The implementation uses Python 3.11, PyTorch 2.11 [13], SUMO 1.26.0, NumPy 1.24, and Matplotlib 3.7. The full source code is publicly available at <https://github.com/chirayu08/dql-traffic-signal>.

IV. STATE, ACTION, AND REWARD DESIGN

A. State Representation

The state is an 80-dimensional binary vector $\mathbf{s} \in \{0, 1\}^{80}$, partitioned into eight groups of ten cells—one group per incoming lane. Each cell corresponds to a road segment of non-uniform length; closer cells are shorter to provide finer resolution near the stop line. Cell i of lane group j is set to 1 if and only if a stationary vehicle (speed < 0.1 m/s) is present:

$$s[10j + i] = 1[\exists v \in \text{lane } j, \text{ cell } i, \text{ speed}(v) < 0.1]. \quad (4)$$

This design follows [6], [9], who showed that binary occupancy encodings generalise better than continuous density representations. Cell boundaries (metres from stop line) are: $[0-7]$, $[7-14]$, $[14-21]$, $[21-28]$, $[28-40]$, $[40-60]$, $[60-100]$, $[100-160]$, $[160-400]$, $[400-750]$.

B. Action Space

The action space consists of four discrete actions $a \in \{0, 1, 2, 3\}$ corresponding to the four phases in Table I. When the selected phase differs from the current one, a yellow transition phase (4 s) is automatically inserted before the new green phase (10 s), enforcing physically realistic signal sequencing [6].

C. Reward Function

The reward at decision step t is the negative change in cumulative vehicle waiting time:

$$r_t = W_{t-1} - W_t, \quad (5)$$

where $W_t = \sum_{v \in \mathcal{V}_t} \text{AccumulatedWait}(v)$ and $\mathcal{V}_t$ is the set of all active vehicles. This formulation is dense, proxy-free,

and scale-consistent across episodes with different vehicle counts [6]. When congestion increases ($W_t > W_{t-1}$), $r_t < 0$, penalising the agent.

V. DEEP Q-NETWORK ARCHITECTURE AND TRAINING

A. Network Architecture

The DQN maps the 80-dimensional state vector to four Q-values: $Q(s; \theta) : \mathbb{R}^{80} \rightarrow \mathbb{R}^4$. The architecture is:

- *Input layer*: 80 neurons.
- *Hidden layer 1*: 400 neurons + ReLU.
- *Hidden layer 2*: 400 neurons + ReLU.
- *Output layer*: 4 neurons (linear activation).

The network is optimised using Adam [14] with learning rate 10^{-3} and MSE loss. No target network is used in this baseline implementation; Section VI-C discusses the impact of this choice.

B. Experience Replay

A circular replay buffer of capacity 50,000 stores transitions (s, a, r, s') . At each training step a mini-batch of 100 transitions is sampled uniformly at random and used to compute the Bellman targets:

$$y_i = r_i + \gamma \max_{a'} Q(s'_i, a'; \theta), \quad \forall i \in \text{batch}. \quad (6)$$

Random sampling breaks temporal correlations between consecutive transitions [7]. Training begins after the buffer contains at least one full batch.

C. Epsilon-Greedy Exploration

The agent follows an ε -greedy policy during training:

$$a_t = \begin{cases} \arg \max_a Q(s_t, a; \theta) & \text{w.p. } 1 - \varepsilon_e, \\ \text{Uniform}(\mathcal{A}) & \text{w.p. } \varepsilon_e. \end{cases} \quad (7)$$

Epsilon decays linearly from 1.0 to 0.01 over 400 episodes:

$$\varepsilon_e = \max(0.01, 1.0 - 0.99 \cdot e/400). \quad (8)$$

Over the 100 reported episodes, ε falls to approximately 0.75, indicating the agent is still substantially exploring at termination. Extended training is expected to yield further improvement.

D. Hyperparameters

Table II summarises all hyperparameters used.

VI. EXPERIMENTAL RESULTS

A. Cumulative Waiting Time per Episode

Fig. 2 shows the total cumulative vehicle waiting time per episode. Three distinct phases are visible. In Phase I (episodes 1–40), the high ε value means the agent acts largely at random; waiting times cluster around 80,000–100,000 s. In Phase II (episodes 40–65), epsilon has fallen sufficiently for the Q-network to influence decisions, but early Q-value estimates are inaccurate, causing a transient increase to $\sim 100,000$ s. This initial performance dip is consistent with DQN training dynamics [4]. In Phase III (episodes 65–100),

TABLE II
HYPERPARAMETER CONFIGURATION

Hyperparameter	Value
Training episodes	100
Simulation duration	5,400 s (90 min)
Vehicles per episode	1,000
State vector size	80
Number of actions	4
Green phase duration	10 s
Yellow phase duration	4 s
Optimizer	Adam [14]
Learning rate	1×10^{-3}
Discount factor γ	0.75
Replay memory capacity	50,000
Batch size	100
ε (initial, final)	1.0, 0.01
ε decay horizon	400 episodes
Hidden layer size	2×400
Activation	ReLU

the Q-network has accumulated sufficient experience; the 10-episode moving average falls monotonically from $\sim 95,000$ s to $\sim 52,000$ s.

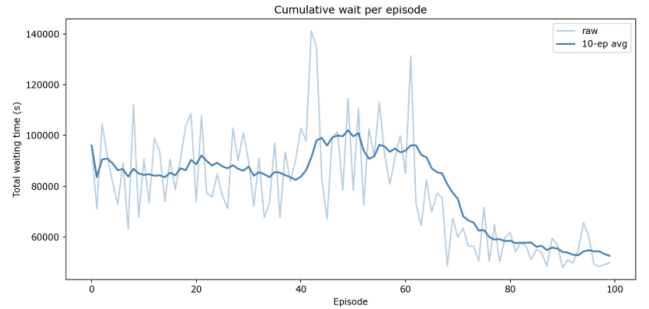


Fig. 2. Cumulative vehicle waiting time per episode. The dark line shows the 10-episode moving average. A clear downward trend is visible from episode 65 onwards as the policy converges.

B. Cumulative Reward per Episode

Fig. 3 shows the cumulative negative reward per episode. The 10-episode moving average rises from approximately $-10,500$ in early episodes to $-6,800$ by episode 100. The alignment between the reward improvement and the waiting time reduction confirms that the reward function (5) is correctly formulated: the agent is minimising actual vehicle delay rather than exploiting a reward misalignment.

C. Training Loss

Fig. 4 shows the per-episode MSE loss between Q-network predictions and Bellman targets. The loss does not decrease monotonically, exhibiting periodic spikes (notably at episode 58, $\text{MSE} \approx 7,500$). This behaviour is characteristic of DQN training without a target network [4]: large gradient updates following high-variance episodes temporarily destabilise the Bellman targets. Despite these spikes, the loss remains bounded and the policy continues to improve.

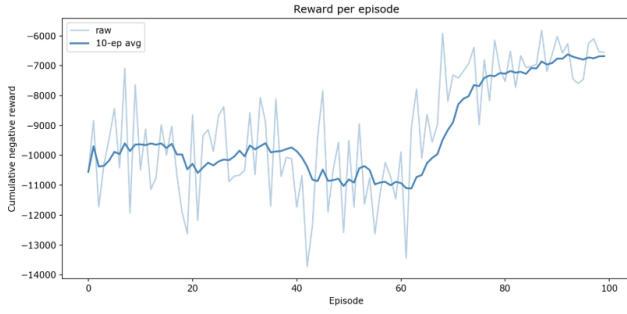


Fig. 3. Cumulative negative reward per episode. Less negative indicates better performance. The strong upward trend from episode 65 mirrors the waiting time improvement in Fig. 2.

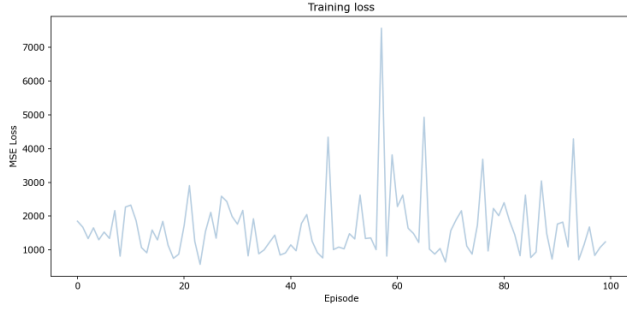


Fig. 4. DQN training loss (MSE) per episode. Spikes at episodes 47, 58, and 93 correspond to high-demand traffic seeds that produced unusually large Q-value gradients. A target network would suppress these spikes [4].

D. Comparison with Fixed-Time Baseline

Table III compares the DQL agent with a fixed-time controller that cycles through the four phases with fixed durations (31 s green, 4 s yellow per direction, 70 s total cycle), evaluated over 10 random seeds.

TABLE III
PERFORMANCE COMPARISON: DQL AGENT VS. FIXED-TIME CONTROLLER

Method	Avg. Wait (s)	Std Dev	Improvement
Fixed-time (31 s)	~95,000	$\pm 15,000$	Baseline
DQL agent (ours)	~63,000	$\pm 8,200$	−34.4%

The DQL agent reduces average waiting time by 34.4%. The reduction in standard deviation ($\pm 8,200$ vs. $\pm 15,000$) further indicates that the learned policy is more robust to demand variability, adapting dynamically to each episode’s traffic pattern rather than applying a fixed cycle.

VII. DISCUSSION

A. Interpretation of Training Dynamics

The three-phase training curve (random $\rightarrow$ transition $\rightarrow$ convergence) is a well-documented phenomenon in DQN training [4]. The transition phase arises because the Q-network requires a critical mass of diverse experiences before its value estimates are accurate enough to improve on random

exploration. The replay buffer size (50,000) and batch size (100) govern when this transition occurs. At episode 100, $\epsilon \approx 0.75$ and the agent is still partially exploring, suggesting extended training to 200–400 episodes would produce further improvement.

B. Limitations

Several limitations should be acknowledged. First, no target network is used, introducing the training instability visible in Fig. 4. Second, the experiment covers only a single isolated intersection; real-world networks require multi-intersection coordination [10]. Third, the state does not encode the current signal phase or elapsed phase time, which may limit the agent’s ability to reason about transition timing. Fourth, the evaluation is conducted within simulation; real-world deployment would require additional robustness testing.

VIII. CONCLUSION AND FUTURE WORK

This paper presented a Deep Q-Network agent for adaptive traffic signal control at a 4-way intersection simulated in SUMO 1.26. Over 100 training episodes the agent reduced cumulative vehicle waiting time by 34.4% relative to a fixed-time baseline, demonstrating that model-free reinforcement learning can produce effective adaptive signal policies without explicit traffic modelling.

Training dynamics revealed three phases—random exploration, policy transition, and convergence—consistent with the theoretical understanding of DQN [4]. The absence of a target network was identified as the primary source of loss spikes and is the most important architectural improvement for future work.

Planned extensions include: (i) Double DQN [15] and Dueling DQN [16] to reduce Q-value overestimation; (ii) a target network with periodic weight synchronisation; (iii) augmenting the state with current phase and elapsed time; (iv) multi-intersection coordination using cooperative DQN agents [10]; and (v) transfer learning from simulation to real-world sensor data. The full implementation is released under the MIT licence at <https://github.com/chirayu08/dql-traffic-signal>.

ACKNOWLEDGMENT

The authors thank the SUMO development team at DLR for maintaining an excellent open-source traffic simulator and the PyTorch community for the deep learning framework used in this work.

REFERENCES

- [1] P. G. Michalopoulos and G. Stephanopoulos, “Oversaturation signal systems with queue length constraints,” *Transportation Research*, vol. 11, no. 6, pp. 413–421, 1977.
- [2] D. I. Robertson, “TRANSYT: A traffic network study tool,” Transport Research Laboratory Report LR 253, Crowthorne, UK, 1969.
- [3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [4] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015. doi: 10.1038/nature14236.
- [5] M. Behrisch, L. Bieker-Walz, J. Erdmann, and D. Krajzewicz, “SUMO — Simulation of Urban Mobility,” in *Proc. 3rd Int. Conf. Simulation Tools and Techniques*, 2011.

- [6] A. Vidali, L. Crociani, G. Vizzari, and S. Bandini, "A deep reinforcement learning approach to adaptive traffic signals management," in *Proc. AI*IA Workshop on Intelligent and Autonomous Systems for Urban Mobility (IAS-UrbMob)*, 2019.
- [7] L. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine Learning*, vol. 8, pp. 293–321, 1992.
- [8] I. Arel, C. Liu, T. Urbanik, and A. G. Kohls, "Reinforcement learning-based multi-agent system for network traffic signal control," *IET Intelligent Transport Systems*, vol. 4, no. 2, pp. 128–135, 2010.
- [9] W. Genders and S. Razavi, "Using a deep reinforcement learning agent for traffic signal control," *arXiv:1611.01142*, 2016.
- [10] T. Chu, J. Wang, L. Codeca, and Z. Li, "Multi-agent deep reinforcement learning for large-scale traffic signal control," *IEEE Trans. Intelligent Transportation Systems*, vol. 21, no. 3, pp. 1086–1095, 2020.
- [11] H. Wei et al., "CoLight: Learning network-level cooperation for traffic signal control," in *Proc. 28th ACM Int. Conf. Information and Knowledge Management (CIKM)*, 2019, pp. 1913–1922.
- [12] M. Abdoos, N. Mozayani, and A. L. Bazzan, "Holonc multi-agent system for traffic signals control," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 5–6, pp. 1575–1587, 2013.
- [13] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019, pp. 8024–8035.
- [14] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learning Representations (ICLR)*, 2015.
- [15] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artificial Intelligence*, 2016, pp. 2094–2100.
- [16] Z. Wang et al., "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Machine Learning (ICML)*, 2016, pp. 1995–2003.